

Refatoramento de Código

Exemplo extraído do livro *Refactoring: Improving the Design of Existing Code* (Fowler, 1999)

Parte 7: Exercício Concluído!!!

I. Análise sobre as mudanças realizadas

Após realizar todas as mudanças guiadas e executar o código-fonte atual, perceba que, em tempo de execução, podemos modificar a classificação de um DVD; basta chamar o método `setCódigoDePreço()`, como antes.

Os objetos concretos de classificação devem implementar `getValorDaLocacao()` e `getPontosDeAlugadorFrequente()` que estão em **DVD**. Aqui, mais uma vez, teremos **delegação** da responsabilidade para um objeto concreto de classificação. Os métodos citados serão chamados polimorficamente.

Comparando com o código inicial apresentado na etapa 1.2.1, será que valeu a pena tanto esforço para chegarmos ao ponto de introduzir o **polimorfismo**? Vamos trazer algumas ponderações:

- Primeiramente, um programador com razoável visão de projeto de software já teria pensado em introduzir polimorfismo desde o início do projeto;
- Em segundo lugar, valeu a pena sim: o código é muito mais simples de mudar quando houver um novo esquema de preços ou regra de pontos de alugador frequente, por exemplo.

II. Desfecho: Herança x Composição

Composição e **herança** são dois mecanismos utilizados para reutilizar funcionalidades. Alguns anos atrás (e na cabeça de alguns programadores ainda!), a **herança** era considerada a ferramenta básica de extensão e reutilização de funcionalidade. A **composição** estende uma classe pela delegação de trabalho para outro objeto. A **herança** estende atributos e métodos de uma classe.

Hoje, considera-se que a **composição** é muito superior à **herança** na maioria dos casos. A **herança** tem sua aplicabilidade em alguns contextos. O presente exercício de refatoramento é um excelente exemplo do uso da **composição** em vez da **herança**.

As grandes vantagens da **composição** são:

- A **herança** gera um acoplamento muito forte. A **composição** pode reduzir acoplamento, principalmente se interfaces forem usadas;

- A **composição** pode ser mudada em tempo de execução.

A grande vantagem da **herança** é que ela gera código mais simples de ler e entender:

- A **composição** usa muitos objetos pequenos e só sabemos quem vai ser composto com quem em tempo de execução; isso é mais difícil de entender.

Vejamos agora, como ficou o diagrama de classes final deste exercício de refatoramento:

